



GBS SE Agent, explained in plain English

What we built, how it works, where its limits are, how it compares to the big names — and what the next two steps would give us. No jargon, everyday examples, pictures.

Written for	Anyone who has to understand or decide about this, technical or not
About	GBS SE Agent v0.0.12, the coding assistant inside VS Code
Date	5 October 2026

PART ONE

What is this thing, really?

If you remember one thing: it is a very fast, very literal junior assistant who works inside the code, and shows you everything it does.

Imagine hiring someone on their first day. They don't know where anything is. But they are quick, they never get bored, and they have three good habits:

- **They look before they touch.** They don't guess where the login code lives — they go and find it.
- **They ask before anything risky.** Deleting things, running commands, opening a file that looks like it holds a password — they stop and check with you first.
- **They show their receipts.** When they're done, they tell you exactly which files they touched, so you can look before anything goes out.

That is the whole product. Everything technical in this document exists to force those three habits onto a computer program that would otherwise happily guess.

THE ONE-SENTENCE VERSION

A helper that reads and changes our code on its own, shows every step, asks permission before anything it can't undo, and can run entirely on our own computers so client code never leaves the building.

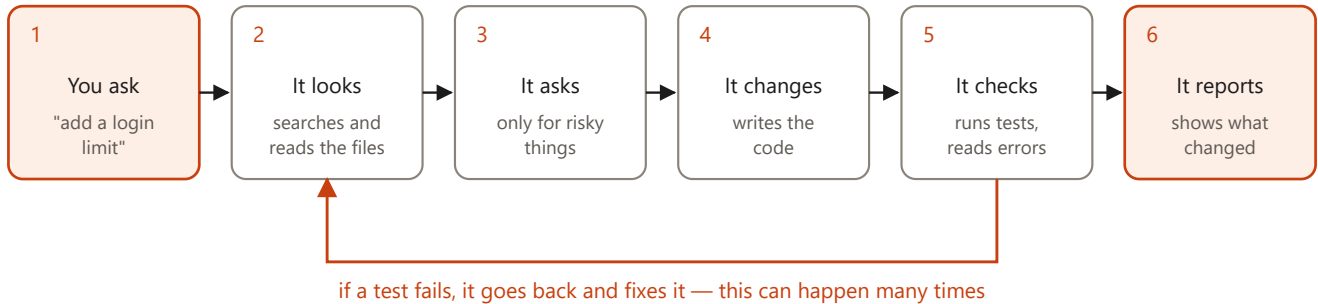
What it is not

It is not a chatbot that answers questions. A chatbot talks; this one *acts* — it opens files, edits them, runs the tests, reads the errors and tries again. And it is not a replacement for a developer: it is very good at the boring, repetitive half of the job and genuinely unreliable at deciding what should be built.

PART TWO

How it works, step by step

One instruction from you turns into a loop of small steps. The loop is the whole trick.



Steps 2 to 5 repeat until the work is actually done. That is what makes it an "agent" rather than something that gives you an answer and stops.

One instruction, many small steps. A simple edit is 3–8 steps; a big change can be 20–60.

WHY THIS MATTERS FOR COST AND SPEED

Every one of those steps is a separate question to the AI. So when someone says "the free plan allows 1,000 requests a day", that is not 1,000 tasks — it's more like 100. This is the single most misunderstood number in the whole subject.

PART THREE

Three words you'll keep hearing

Model, tokens, context window. Everything else is detail.

The model is the brain

The agent itself has no intelligence. It is the hands, the eyes and the rulebook. The thinking is done by a separate thing called a **model** — and we can plug in different brains: one running on your own laptop, one on a company server, or one rented over the internet. Same agent, different brain, very different speed and cost.

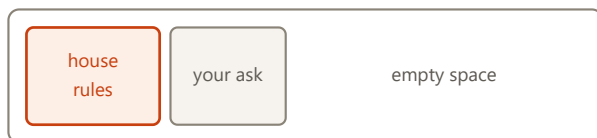
Tokens are pieces of words

AI models don't read letters or words; they read **tokens**, which are chunks of about four characters. "Understanding" might be three tokens. A rough rule: **750 words $\approx$ 1,000 tokens**. You pay for tokens, and the brain can only hold so many at once.

The context window is the desk

Picture a desk that fits exactly 40 sheets of paper. That's the **context window**. Everything the assistant is currently thinking about has to be on that desk: the house rules, your instruction, every file it has read, every test result. When the desk fills up, something has to go.

THE DESK AT THE START — PLENTY OF ROOM



AFTER 20 STEPS — NEARLY FULL



↓ the desk is full — now what?

WHAT OUR AGENT DOES: TIDIES THE DESK



Our agent clears space in a fixed order: first it throws away old tool output, then older parts of the conversation, and only then asks the AI to write a summary. The house rules never move — that part is what makes it fast.

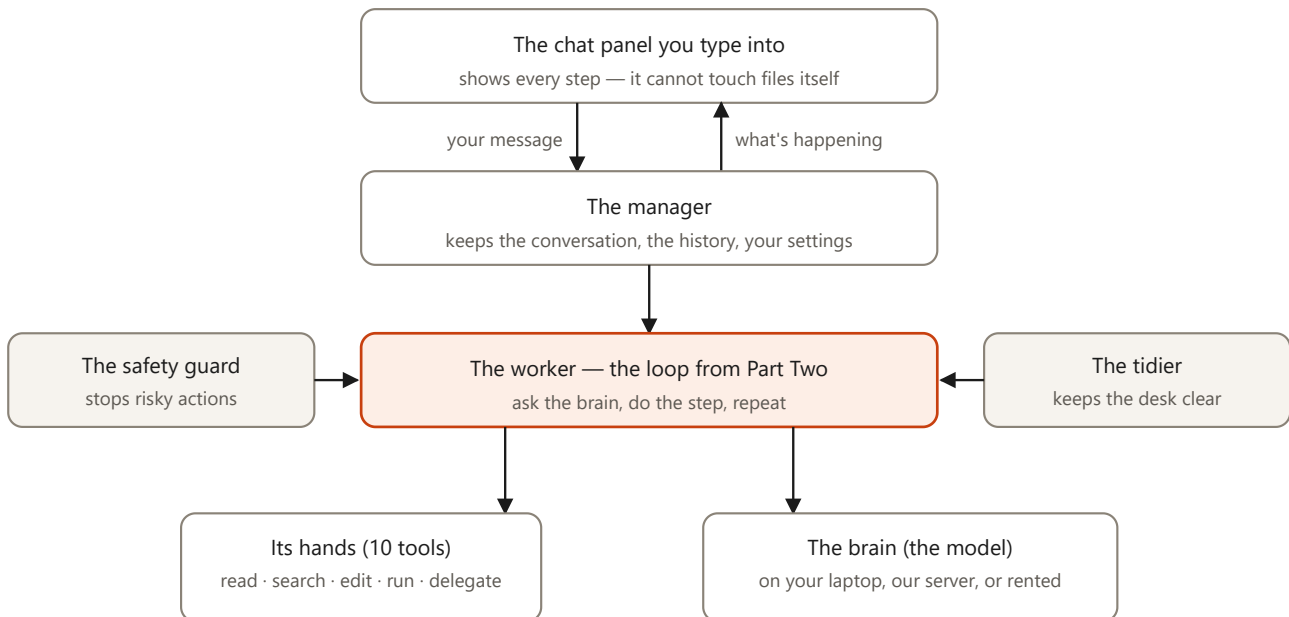
IN NORMAL NUMBERS

Our default desk is 32,768 tokens — roughly 24,000 words, or about 50 pages. A big model on a good server can have a desk four times that size. But a bigger desk is not automatically better: models get noticeably worse at finding things in the middle of a very full desk, which is why we tidy it even when there's room.

PART FOUR

What's inside our agent

Four parts, each with one job. Nothing skips a layer, which is why it's safe.



The chat panel is deliberately powerless — it can only show things and pass on what you type. Anything that touches a real file goes through the safety guard first.

Two clever bits worth knowing

Skills. A folder where we write down our house rules — "always use our DataGrid, never a plain table". The assistant only reads the rules when the job actually involves that topic, so they cost us almost nothing the rest of the time. Like a recipe book on the shelf rather than read aloud at every meal.

Helpers. For big searches, the assistant sends a junior helper who reads a hundred files in its own room and comes back with a three-line answer. The hundred files never land on the main desk. This is why it can work on a large codebase with a small desk.

PART FIVE

Where the brain lives — and what that costs

Three options. They trade privacy, speed and money against each other.

1. On your laptop

Like cooking at home

Free, works on a plane,
nothing ever leaves

Slower, and the brain
has to be small enough
to fit your machine

2. On a company server

Like a staff canteen

Big brain, fast, shared
by the whole team,
code stays with us

Costs real money up
front and needs setup

3. Rented online

Like a takeaway

Strongest brains, no
hardware to buy,
free tiers to try

Our code goes to
someone else's computer

The agent works with all three, and you switch with two clicks. Option 1 is the one nobody else makes easy — it's our distinguishing feature.

Why the rented one sometimes says "server overload"

When you rent, you share. A free plan is a busy shop at lunchtime: sometimes the answer is "too many people right now, come back in a minute". That is the `429` error we saw. It isn't a bug in our code or a problem with your key — it's a queue.

Until last week, one of those messages stopped the whole job. Now the agent waits a second and tries again, then two seconds, then four — and tells you it's waiting instead of looking frozen. The same thing a polite person does when the line is engaged.

If we run our own server

This is buying the pizza oven instead of ordering delivery. It makes sense at volume, and the sums are about **memory on the graphics card**, not about speed. Each conversation the brain is holding takes up memory, and a bigger desk takes proportionally more. A mid-size brain with a very large desk can use 30–40 GB of card memory *per person using it at that moment*.

THE ONE FREE WIN

Our agent sends the same house rules at the start of every single step. Server software can be told to remember that repeated opening — it's called prefix caching — and it makes every step after the first much faster, for the price of one setting. We should turn it on from day one.

PART SIX

The speed limiters inside

We put safety caps on the agent. They were set for small brains, and a better brain makes some of them too tight.

Think of a hire car with a speed limiter set for a learner driver. Sensible for a learner, frustrating for someone who knows the road. Our agent has about a dozen of these, and they do real jobs: they stop a runaway job burning money, stop one giant file flooding the desk, stop an infinite loop.

THE LIMIT	IN PLAIN ENGLISH	WHAT HAPPENS WITH A BETTER BRAIN
Desk size 32,768	How much it can think about at once	Can be 4× bigger — the main reason to upgrade
Steps per job 60	How many small steps before it stops and checks in	Too few for big jobs; should be 120+
Answer length 8,192	How much it can write in one go	Long files get cut off; should be doubled
Tool output 20,000 characters	How much of a test result or search it keeps	Failures get truncated right where it matters
Lines read at once 1,000	How much of a file it opens in one go	A big brain could read whole files
Helpers at once 4	How many junior helpers can work in parallel	Our own server could handle many more
Patience 5 minutes	How long it waits for the brain to start replying	Will bite us first — a big brain waking up can take longer, and the job dies

Six of these are adjustable in Settings today. The rest are fixed in the code, which means someone with a better brain cannot get the value out of it. **The proposal is to make the handful that depend on the brain adjustable too** — about half a day of work, with every current value kept as the default, so nothing changes for anyone who doesn't touch them.

WHY NOT MAKE EVERYTHING ADJUSTABLE?

Because every switch is a switch someone can set wrongly, and then we get the support call. The rule we'd follow: if the right value depends on which brain you use, it becomes a setting. If it's an internal balancing act nobody can reason about, it stays ours to tune.

PART SEVEN

How we compare to the big names

Honestly: we win on privacy and control, we lose on polish and connections.

CAN IT...	OURS	CLINE / COPILOT / CURSOR	WHAT IT MEANS DAY TO DAY
Work fully offline, free	Yes	Partly / No	Client code never leaves the building — our biggest advantage
Ask before risky actions	Yes, strongly	Yes	Ours is stricter than most, which suits regulated clients
Learn our house rules	Yes (skills)	Yes	We're level with the leaders here
Undo everything it did	No	Yes	We show differences, but there's no one-click rewind
Point at a file: "use @this"	No	Yes	The most-missed everyday convenience
Plug into Figma, Jira, databases	No	Yes (MCP)	The biggest gap — see Part Eight
Finish your line as you type	No	Yes	A different product really; we're the "do the task" tool
Understand a screenshot	No	Yes	Can't paste a picture of a broken screen yet

THE FAIR SUMMARY

For a company that cannot send client code to the cloud, ours is a genuinely good answer and the others are not. For a developer who just wants the smoothest experience and doesn't care where the code goes, the others are ahead. Those two sentences should drive what we build next.

PART EIGHT

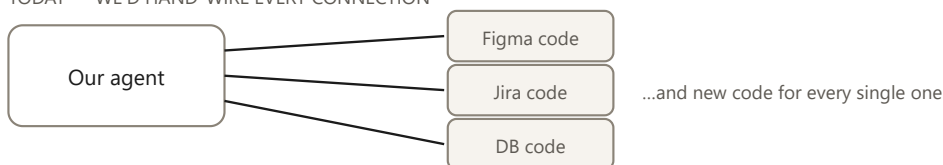
The next big idea: a universal plug

MCP is a standard socket. Build it once, and the assistant can talk to anything that has the matching plug.

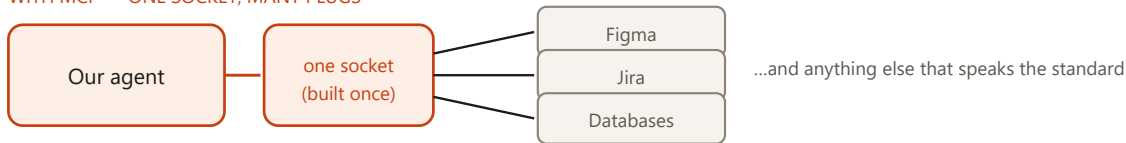
Right now our assistant can do exactly ten things, all about files and commands. If we wanted it to read a design from Figma, we'd have to write Figma-specific code. Then Jira-specific code. Then database-specific code. Forever.

MCP is the agreed shape of the plug. It's the USB of AI tools: Figma, Jira, databases and hundreds of others already publish one. We build the socket once, and all of them work.

TODAY — WE'D HAND-WIRE EVERY CONNECTION



WITH MCP — ONE SOCKET, MANY PLUGS



The win isn't Figma specifically — it's that every future connection becomes a config line instead of a development project.

What Figma would actually give us — and what it wouldn't

It will **not** build a screen on its own. What it does is let the assistant read the real design instead of guessing: the exact spacing, the real colour names, which component the designer used. Combine that with our house rules — "when the design says Button/Primary, use our Button component" — and you get code that matches both the design and our standards. That pairing is something none of our competitors can offer a GramPro client out of the box.

Honest cost: roughly one to two weeks for a first useful version, and it must be built carefully — a plug defined by a project we open is, in effect, permission to run a program on our machine. The same "ask once, clearly" approach we already use for house rules applies.

PART NINE

What I'd do next, in order

Two small things that unlock the hardware we're buying, then the ambitious one.

ORDER	WHAT	EFFORT	WHY NOW
1	Make the fixed limits adjustable	Half a day	Without it we'd buy a bigger brain and keep driving at learner speed
2	One-click undo of the agent's work	2–3 days	The thing that makes people brave enough to let it run on its own
3	"Use @this file" shortcuts	2–3 days	Removes the most common daily annoyance
4	The universal plug (MCP)	1–2 weeks	Closes the biggest gap, and opens the Figma + house-rules demo

Items 1–3 are small, safe and immediately felt. Item 4 is the one that changes what the product *is*, and it deserves a written design before any code, because the two risky parts — how much room the new tools take on the desk, and trusting plugs defined by a project — are decisions, not typing.

A glossary, if someone quotes jargon at you

Model

The brain. We rent one or run our own; the agent is the body around it.

Token

A chunk of about four characters. 750 words is roughly 1,000 tokens.

Context window

The desk. How much the brain can hold at once.

Tool calling

The brain being able to say "read this file" instead of only talking.

Rate limit / 429

"Too many people right now, try again shortly."

MCP

The standard plug shape that lets the agent talk to Figma, Jira, databases.

Local model

A brain running on our own hardware. Nothing leaves the building.

Prefix caching

The server remembering the repeated opening of each request, so it's faster.

GBS SE Agent v0.0.12 · 26 automated tests passing · runs on Ollama, LM Studio, llama.cpp, vLLM, Hugging Face, Groq or Google AI Studio. Written 5 October 2026. This document is not committed to the repository.